

SliceGX: Layer-wise GNN Explanation with Model-slicing

Cibo Yu
Zhejiang University
Hangzhou, China
cbyu@zju.edu.cn

Tingting Zhu
Zhejiang University
Hangzhou, China
tingtingzhu@zju.edu.cn

Tingyang Chen
Zhejiang University
Hangzhou, China
chenty@zju.edu.cn

Yinghui Wu
Case Western Reserve University
Cleveland, USA
yxw1650@case.edu

Arijit Khan
Bowling Green State University
Bowling Green, USA
arijitk@bgsu.edu

Xiangyu Ke*
Zhejiang University
Hangzhou, China
xiangyu.ke@zju.edu.cn

Abstract

Ensuring the trustworthiness of graph neural networks (GNNs), which are often treated as black-box models, requires effective explanation techniques. Existing GNN explanations typically apply input perturbations to identify subgraphs that are responsible for the occurrence of the final output of GNNs. However, such approaches lack finer-grained, layer-wise analysis of how intermediate representations contribute to the final result, capabilities that are crucial for model diagnosis and architecture optimization. This paper introduces SliceGX, a novel GNN explanation approach that generates explanations at specific GNN layers in a progressive manner. Given a GNN model $\mathcal{M}$, a set of selected intermediate layers, and a target layer, SliceGX slices $\mathcal{M}$ into layer blocks (“model slice”) and discovers high-quality explanatory subgraphs within each block that elucidate how the model output arises at the target layer. Although finding such layer-wise explanations is computationally challenging, we develop efficient algorithms and optimization techniques that incrementally construct and maintain these subgraphs with provable approximation guarantees. Extensive experiments on synthetic and real-world benchmarks demonstrate the effectiveness and efficiency of SliceGX, and illustrate its practical utility in supporting model debugging.

CCS Concepts

• Computing methodologies → Neural networks; • Mathematics of computing → Graph algorithms.

Keywords

Graph neural networks, Explainable AI, Model-slicing

ACM Reference Format:

Cibo Yu, Tingting Zhu, Tingyang Chen, Yinghui Wu, Arijit Khan, and Xiangyu Ke. 2026. SliceGX: Layer-wise GNN Explanation with Model-slicing. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792689>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, April 13–17, 2026, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792689>

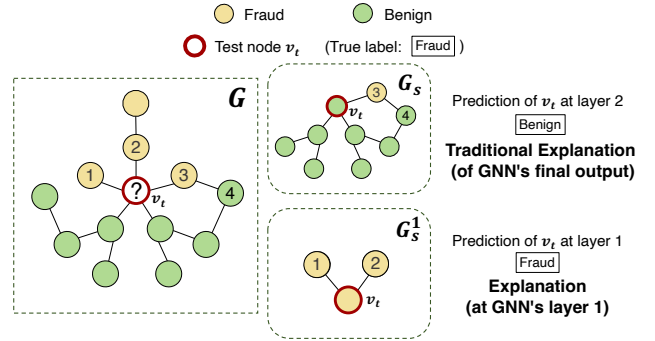


Figure 1: Generating layer-wise explanations for GNN diagnosing for spam review detection [12].

1 Introduction

Graph Neural Networks (GNNs) have demonstrated promising performances in various applications, such as recommender systems [7], knowledge graph reasoning [50], and social network analysis [32]. A GNN is a model $\mathcal{M}$ that converts an input graph G and its initial node feature matrix Z^0 into a new numerical matrix Z , which can be further post-processed to produce task-specific answers, such as node classification and graph classification.

However, predictive accuracy alone does not guarantee the trustworthiness and reliability of graph learning; transparency of the inference process is also required [3]. Understanding how a GNN arrives at a decision is therefore essential for model provenance and tracking. For example, a provenance-oriented analysis [3] may ask “What/Which” questions to identify which input features are relevant, “How” questions to trace how features transform across layers, or “Where/When” questions to determine which layers contribute most during inference. Such finer-grained, “layer-wise” interpretation, as also practiced in other deep models, e.g., in CNNs [22] and DNNs [36], further benefits model optimization and debugging [2].

Despite the evidential need, the study of layer-wise interpretation for GNNs, and how it benefits GNN applications, still remains in its infancy. Consider the following example.

EXAMPLE 1. A fraction of a spam review network G [12] is illustrated in Figure 1, where nodes are reviews and edges indicate associations such as shared reviewers, common products, or temporal links. A GNN classifies reviews as either “fraud” (yellow) or “benign” (green), with fraud reviews exhibiting deceptive or spam-like behavior.

Observing that a fraud review v_t is incorrectly labeled as “benign”, the analyst may want to diagnose the inference process of the GNN $\mathcal{M}$. Several “diagnose” queries, analogous to those in CNNs and DNNs [22, 36], can be posed:

- “Q1: At which layer (When) did $\mathcal{M}$ first err?” Identifying the first erroneous layer enables “stop-on-first-error” inspection and early intervention.;
- “Q2: Which fraction of G (subgraphs) are responsible for this first mistake?” This reveals the nodes, edges, and features that directly influence the decision;
- “Q3: How did the influence propagate via subsequent layers?” This traces how the identified influences evolve and spread during inference.

A closer inspection of $\mathcal{M}$ ’s inference process reveals the following.

(1) For Q1, by treating each layer as an “output layer”, we can “segment” the inference process and analyze the node embedding of v_t at a specific layer to assess if the accumulated neighborhood impact leads to an incorrect label. The established strategy that aggregates localized neighbors such as “soft labels” [41] to track node influence also justifies this intuition. For example, v_t is correctly classified as “fraud” at layer 1 but is misclassified as “benign” at layer 2, alerting a first error at layer 2 for further investigation.

(2) For Q2, existing GNN explainers [17, 20, 33, 43, 44] such as GNNExplainer [43] (see Related Work) provide factual or counterfactual subgraph explanations that are “faithful” to the label. However, they typically focus on the final output and do not adapt naturally to intermediate layers as derived for Q1. The influential subgraph at layer 1, G_s^1 , can differ substantially from that at layer 2, G_s (in Figure 1), reflecting distinct local influences and feature structures that explain the layer-specific predictions.

(3) To answer Q3, one needs to also necessarily annotate the explanation structures by distinguishing between influencing nodes and edges that contribute most to the output at targeted layers and those that are only “connecting” and merely propagating such influence during the inference. For example, the graph G_s may indicate that the first mistake is due to accumulated influence from a majority of “benign” 2-hop neighbors of v_t at layer 2, while an adjacent edge of v_t merely propagates this influence (e.g., node 3 propagates information from node 4), hence suggesting further investigation for the “How” question Q3. Prior GNN explainers generally do not separate these roles.

The above example calls for a layer-wise explanation framework for GNNs that can efficiently produce concise, consistent explanations for intermediate-layer outputs during inference. We aim to address the following questions: (1) How to define explanation structures for layer-wise GNN explanations? (2) How to measure the explainability of layer-wise explanations? and (3) How to efficiently generate layer-wise GNN explanations? Existing GNN explainability methods do not fully answer these questions.

Our main contributions are as follows.

- We extend GNN explanation to its layer-wise counterpart. To this end, we introduce the “model slice” concept and a novel explanatory subgraph structure, which together enable precise analysis of a GNN’s layer-wise influence.
- We present SliceGX, a layer-wise GNN explanation framework to generate explanatory subgraphs at different layers.

To guide the generation process, we propose a bi-criteria quality measure that integrates message-passing influence and embedding diversity.

- We present training-free post-hoc explanation generation algorithms to discover connected subgraph explanations for specific model layers. We propose an efficient $\frac{1}{2}$ -approximation generation algorithm for single-node single-source layers and generalize it to multi-node multi-source settings.
- We validate SliceGX on six benchmarks, demonstrating its efficiency and effectiveness. Crucially, it uncovers GNN insights that facilitate downstream tasks like debugging and optimization.

2 Related Work

Explainability of GNNs. Several approaches have been proposed to explain GNNs’ predictions [17, 20, 21, 33, 43, 44, 47], primarily by identifying important nodes, edges, node features, subgraphs, and message flows. A common strategy is to monitor the changes in prediction under input perturbations. For example, GNNExplainer [43] learns soft masks over edges and node features to maximize mutual information between original and masked graph predictions. SubgraphX [44] efficiently samples explanatory subgraphs using pruning-based Monte Carlo Tree Search (MCTS) together with Shapley-value estimates. In contrast, SAME [42] extends this with a two-phase framework that first identifies multiple important substructures and then aggregates them into a final explanation. Other lines of work use surrogate models to approximate GNN predictions and derive explanations from the surrogate [13, 20].

Despite their strengths, these methods are not directly suitable for layer-wise explanations. First, they typically focus on final outputs, failing to capture the progressive decision formation across layers. Second, they do not distinguish between influence sources and structural propagators. Third, they cannot explicitly explain intermediate-layer outputs.

Layer-wise GNN Analysis. A smaller body of work studies layer-wise attribution for GNNs. Decomposition-based methods, i.e., GNN-LRP [34] and Excitation-BP [29], assign importance scores per node at each layer by backpropagating relevance. GraphMask [33], a post-hoc interpretation method, produces differentiable edge masks for each layer but focuses on explaining the final prediction rather than intermediate targets. FlowX [17] models explanations as message flows, which are sequences of layer edges, allowing it to calculate the importance of specific message-passing steps at each individual layer. While valuable, these methods share a limitation: they interpret intermediate layers relative to the final prediction rather than providing self-contained explanations for intermediate outputs.

Model Slicing. Model slicing, with a counterpart in program slicing [38], extracts sub-models for finer-grained analysis [5] and has been applied to efficient inference [6, 46], adversarial detection [51], mechanistic interpretability [31], and visual analytics [19]. This paper makes a first step to adapt model-slicing to the GNN setting and develop efficient algorithms to generate layer-specific explanations that are both concise and annotated with functional roles.

3 Graph Neural Networks and Explanation

Graphs. A graph $G = (V, E)$ has a set of nodes V and a set of edges E . Each node v carries a tuple, representing its attributes (or features) and their corresponding values.

We define the following terms: (1) The l -hop neighbors of a node $v \in V$, denoted $N^l(v)$, are nodes *within* l hops from v in G . (2) The l -hop neighbor subgraph, $G^l(v)$, is a node induced subgraph of G , which is induced by $N^l(v)$. (3) For a set of nodes $V_s \subseteq V$, the l -hop neighbor subgraph, $G^l(V_s)$, is induced by the union of l -hop neighbors of nodes in V_s , i.e., $\bigcup_{v \in V_s} N^l(v)$. $G^l(v)$ is always connected, while $G^l(V_s)$ may not be. (4) The size of a graph is the total number of nodes and edges, given by $|G| = |V| + |E|$.

Graph Neural Networks. GNNs are a class of neural networks that learn to convert an input graph G and its initial node feature matrix Z^0 to proper representations. We provide an abstract characterization of a GNN $\mathcal{M}$ with L layers.

In general, a GNN can be considered as a composition of two functions: an encoder f_1 and a predictor f_2 .

(1) The encoder f_1 is a composite function $f_1^L \circ f_1^{L-1} \circ \dots \circ f_1^1(G)$. At the l -th layer ($l \in [1, L]$), the function f_1^l uniformly updates, for each node $v \in V$, an input representation from the last layer as:

$$z_v^l := \text{TRANS} \left(\text{AGG}^l \left(z_j^{l-1} \mid j \in N^l(v) \cup \{v\} \right) \right) \quad (1)$$

Here, z_v^l is the feature representation (“embedding”) of node v at the l -th layer (z_v^0 is the input feature of v); N_v is the set of neighbors of node v in G , and AGG^l (resp. TRANS^l) represents an aggregation (resp. transformation) function at the l -th layer, respectively.

(2) Subsequently, a predictor f_2 takes the output embeddings $f_1(G) = Z$ and performs a task-specific post-processing $f_2(Z)$, to obtain a final output (a matrix), denoted as $\mathcal{M}(G)$. Here, f_2 typically includes a multi-layer perceptron (MLP) that takes the aggregated representations Z_v from the encoder f_1 , and produces the final output for the specific task, such as node classification or graph classification. Specifically, given a node $v \in V$, the output of $\mathcal{M}$ at v , denoted as $\mathcal{M}(G, v)$, refers to the entry of $f_2(Z)$ (i.e., $\mathcal{M}(G)$) at node v .

The above step is an abstraction of the inference process of $\mathcal{M}$. Variants of GNNs instantiate different aggregation and transformation functions to optimize task-specific output. A canonical example is the Graph Convolutional Network (GCN) [25], which can be viewed as a first-order spectral approximation that aggregates (normalized) neighbor embeddings and applies a nonlinear transformation, e.g., ReLU, as the transformation operator.

Explanation Structure of GNNs. We start with a notion of explanation. Given a GNN $\mathcal{M}$ with L layers, a graph $G = (V, E)$, and an output $\mathcal{M}(G, v_t)$ to be explained, an *explanation* of $\mathcal{M}(G, v_t)$ is a node-induced, connected subgraph $G_s = (V_s \cup V_c, E_s)$ of G , where

- (1) $V_s \subseteq V$ is a set of *explanatory nodes* that clarify the output $\mathcal{M}(G, v_t)$ ($v_t \in V_s$)
- (2) $V_c \subseteq V$ is a *connector set* induced by explanatory nodes V_s , which is defined as $\bigcup_{v_s \in V_s} C(v_s)$ with $0 \leq |V_c| < |N^L(v_t)|$; here $C(v_s)$ is the set of nodes on the paths from v_s to v_t within $N^L(v_t)$;
- In addition, one of the following conditions holds: (1) $\mathcal{M}(G, v_t) = \mathcal{M}(G_s, v_t)$, i.e., G_s is a “factual” explanation; or (2) $\mathcal{M}(G, v_t) \neq \mathcal{M}(G \setminus G_s, v_t)$, i.e., G_s is “counterfactual”.

Here, $G \setminus G_s$ is a graph obtained by removing the edge set E_s from G , while retaining all nodes. The inference process of $\mathcal{M}$ over G_s (or $G \setminus G_s$) determines if G_s is factual (or counterfactual). The input for inference is always a feature matrix Z^0 and an adjacency matrix, regardless of graph connectivity.

Remarks. The above definition is well justified by established GNN explainers, which generate (connected) subgraphs that are factual [43, 44], counterfactual [4], or both [9], as variants of the above definition. The main difference is that we explicitly distinguish explanatory nodes and connect sets. The added expressiveness allows us to track the varying set of “influencing” nodes and edges at different layers (as will be discussed in Section 4.1). This is not fully addressed by prior GNN explainers.

The rest of the paper makes a case for GCN-based node classification. For simplicity, we refer to “class labels” as “labels”, and “GCN-based classifiers” as “GCN”. Our approach *applies to other representative GNNs*, as verified in the Appendix E.

4 SliceGX: Layer-wise Explanations

4.1 Extending Explanations with Model-Slicing

We start with a notation of the model slice.

Model Slices. To gain insights into the inner-working of GNNs and to enable a more granular interpretation, inspired by model slices [5], we create “model slices” of a trained GNN $\mathcal{M}$ with selected individual layers.

Given a GNN $\mathcal{M}$ as a stack of an L -layered encoder f_1 and a predictor f_2 , and a selected layer $l \in [1, L]$, an *l -sliced model* of $\mathcal{M}$, denoted as $\mathcal{M}^l$, is a GNN as a stack of an encoder f_1^l and the predictor f_2 , where f_1^l is a composite function.

$$Z^l = f_1^l \circ f_1^{l-1} \circ \dots \circ f_1^1(G) \quad (2)$$

The inference process of an l -sliced model $\mathcal{M}^l$ computes a result consistently by (1) calling a composition $f_1^l \circ \dots \circ f_1^1(G)$ to generate output embeddings Z^l up to layer l , and (2) deriving $z_{v_t}^l \in Z^l$ and applying $f_2(z_{v_t}^l)$ to obtain the result $\mathcal{M}^l(G, v_t)$, for each node $v_t \in V_T$. In our abstraction, every function f_1^l ($l \in [1, L]$) and f_2 uniformly takes a $|V| \times d$ embedding matrix as input and produces a matrix of the same size. In practice, we assume that the node features’ dimensionality d remains unchanged across layers.

EXAMPLE 2. Figure 2 (upper part) depicts the architecture of an l -sliced model $\mathcal{M}^l$ derived from a given GNN $\mathcal{M}$ with L layers. The encoder of $\mathcal{M}^l$ includes the first l GNN layers $f_1^1, \dots, f_1^l$. The predictor f_2 in $\mathcal{M}^l$ remains the same as in $\mathcal{M}$. The model parameters in the corresponding modules are shared between $\mathcal{M}$ and $\mathcal{M}^l$.

Explaining GNNs with Model Slicing. We naturally extend GNN explanations with model slicing. Given a node $v_t \in V_T$ with an output $\mathcal{M}(G, v_t)$ to be explained, a connected subgraph G_s^l of G is an *explanation* of $\mathcal{M}(G, v_t)$ at layer l , if one of the following holds:

- (1) The subgraph G_s^l is a factual explanation of $\mathcal{M}(G, v_t)$, that is, $\mathcal{M}(G, v_t) = \mathcal{M}(G_s^l, v_t)$; and also (b) $\mathcal{M}(G, v_t) = \mathcal{M}^l(G, v_t) = \mathcal{M}^l(G_s^l, v_t)$. In this case, we say that G_s^l is a *factual explanation* of $\mathcal{M}(G, v_t)$ at layer l ; or

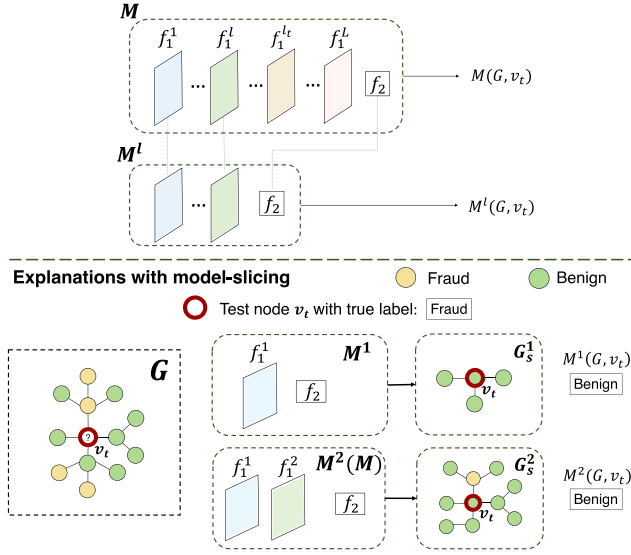


Figure 2: Example of explanations with model-slicing. The upper part depicts the architecture of an l -sliced model M^l , and the lower part shows factual explanations at each layer of M for diagnosing purposes.

(2) The subgraph G_s^l is a counterfactual explanation of $M(G, v_t)$, that is, $M(G, v_t) \neq M(G \setminus G_s^l, v_t)$; and also (b) $M(G, v_t) = M^l(G, v_t) \neq M^l(G \setminus G_s^l, v_t)$. For this case, we say G_s^l is a *counterfactual explanation of $M(G, v_t)$ at layer l* .

Intuitively, we are interested in finding an explanation for an output $M(G, v_t)$, which can be consistently derived by a sliced model at layer l , an “earlier” stage of inference of GNN M . We also provide a detailed discussion in the Appendix A, where we show that this approach yields faithful explanations that align with the model’s true decision-making process.

EXAMPLE 3. Consider a two-layer GNN-based node classifier M as in Example 2 (lower part of Figure 2). The test node v_t in graph $G = (V, E)$ is misclassified as “Benign” by M , and its two “sliced” counterparts M^1 , and M^2 , at layer 1 and layer 2, respectively. Here, G_s^1 and G_s^2 are factual explanations for the incorrect prediction at layers 1 and 2, respectively. While G_s^2 ensures consistency between $M(G, v_t)$ and $M(G_s^2, v_t)$ at the final layer, G_s^1 must maintain consistency in predictions for v_t in both M and the sliced model M^1 . Specifically, M^1 assigns v_t the label “Benign” with G_s^1 , aligning with M ’s final prediction. Inspecting the two explanations, G_s^1 and G_s^2 , one can gain insights by observing the specific fraction of G that may be responsible for an incorrect label “Benign” of node v_t . Better still, the explanations G_s^1 and G_s^2 , putting together, suggest a “chain of evidence” that clarifies the misclassification of v_t by M at early stage (layer 1) and its subsequent layers.

4.2 Explainability Measure

While factual and counterfactual explanations are adopted to characterize “relevant data” that clarify GNN outputs, it is necessary to quantify their explainability for specific diagnosing and interpreting needs. We adopt a novel bi-criteria measure below, in terms of label influence and embedding diversity, each with justification.

Bi-criteria Explainability Measure. Given a graph $G = (V, E)$, a GNN M with L layers, a specific layer number l ($l \in [1, L]$), and a designated node v_t for which the output $M(G, v_t)$ needs to be explained, the *explainability* of an explanation subgraph G_s^l , with the explanatory node set V_s^l , at layer l is defined as:

$$f(G_s^l) = \gamma I(V_s^l) + (1 - \gamma) D(V_s^l) \quad (3)$$

where (1) $I(V_s^l)$ quantifies the relative influence of V_s^l ; (2) $D(G_s^l)$ measures the embedding diversity of V_s^l ; and (3) $\gamma \in [0, 1]$ is a balance factor. Note that the explainability $f(G_s^l) \in [0, 1]$.

We next introduce the functions $I(\cdot)$ and $D(\cdot)$.

Relative influence. We extend GNN feature propagation [9, 48] to layer-wise analysis. We introduce *relative influence*, a measure to evaluate the influence of a node v at a specific layer to a node u with output $M(G, u)$. The relative influence of a node v at layer l on u is quantified by the L1-norm of the expected Jacobian matrix:

$$I_1(v, u) = \left\| \mathbb{E} \left[\frac{\partial Z_u^l}{\partial Z_v^l} \right] \right\|_1 \quad (4)$$

Further, we define the normalized relative influence as $I_2(v, u) = \frac{I_1(v, u)}{\sum_{w \in V} I_1(w, u)}$. We say that the node u is *influenced* by v at layer l , if $I_2(v, u) \geq h$, where h is a threshold for influence. The Jacobian matrix can be approximately computed in PTIME [10, 40].

Given an explanation G_s^l with node set V_s^l , and $N^l(v)$ as l -hop neighbors of $v \in V_s^l$, the *relative influence set* of v is $c(v) = \{u | I_2(v, u) \geq h, u \in N^l(v)\}$. Intuitively, it refers to the nodes that can be “influenced” by a node v in the explanation G_s^l ; the larger, the more influencing node v is.

We next introduce a node set function $I(V_s^l) = \frac{1}{|V_s^l|} \left| \bigcup_{v \in V_s^l} c(v) \right|$.

Given an explanation G_s^l , it measures accumulated influence as the union of $c(v)$ for all $v \in V_s^l$. The larger, the more influential G_s^l is.

Embedding diversity. Optimizing layer-wise explanations in terms of influence analysis alone may still lead to biased, “one-sided” explanations that tend to be originated from the similar type of most influential nodes across layers. We thus consider diversification of explanation generation to mitigate such bias, and enrich feature space [49]. Given an explanation G_s^l at layer l of an output $M^l(G, u)$ of a node u , we define the *diversity set* $r(v)$ as the nodes in $N^l(v)$ with a distance score d above threshold θ , i.e., $r(v) = \{u | d(Z_v^l, Z_u^l) \geq \theta, u \in N^l(v)\}$. The distance function d can be Euclidean distance of node embeddings [26], and $D(V_s^l)$ quantifies diversity as the union of $r(v)$ for all $v \in V_s^l$: $D(V_s^l) = \frac{1}{|V_s^l|} \left| \bigcup_{v \in V_s^l} r(v) \right|$.

Putting these together, we favor explanation with better explainability, which can contribute (1) higher accumulated influence relative to the result of interests from current layers, and (2) meanwhile reveals more diverse nodes on its own, in terms of node embedding. Our method is configurable: other influence and diversity (e.g., mutual information [43]) or explainability measures (e.g., fidelity [15]) can be used to generate explanations for task-specific needs. To demonstrate the poor performance of relying on a single metric, we conducted a detailed ablation study in our experimental section.

EXAMPLE 4. Figure 3 showcases an explanation with high explainability for v_t . The influence set and diversity set of the explanation G_s^l with node set $V_s^l = \{1, 5, v_t\}$ is $\{1, 2, 3, 4, 5, 6, v_t\}$ and $\{1, 2, 3, 5, 6, v_t\}$.

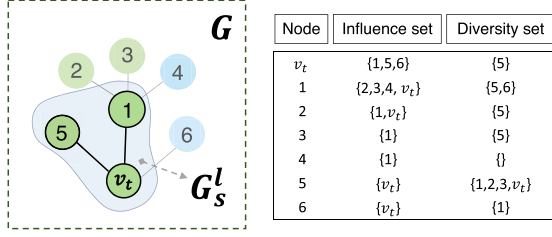


Figure 3: Explanation with high influence and high embedding diversity ($k=3$)

Among all the possible explanations of G size bounded by 3, G_s is favored due to the maximum quality score calculated in Equation 3.

4.3 Explanation Generation Problem

Configuration. We next formulate the problem of explanation generation with model slicing. We characterize the explanation request as a *configuration* $C = (G, \mathcal{M}, v_t, \mathbb{L}, l_t, k)$, as follows:

- a graph G , a GNN $\mathcal{M}$ with L layers, where the l -th layer is indexed with a layer number l ;
- a *target layer* l_t and a *target node* v_t , to specify a designated output $\mathcal{M}^{l_t}(G, v_t)$ to be explained;
- *source layers* $\mathbb{L}$, a set of specified layer numbers, such that for each layer $l \in \mathbb{L}$ ($l \in [1, L]$), an l -sliced model $\mathcal{M}^l$ is derived to provide explanations for $\mathcal{M}^{l_t}(G, v_t)$;
- k is a size bound w.r.t. the number of explanatory nodes.

Given a target result $\mathcal{M}^{l_t}(G, v_t)$ to be explained, the explanation generation with model-slicing is to generate, at each layer $l \in \mathbb{L}$, an optimal explanation G_s^{*l} of G with an explanatory node set V_s , for the output $\mathcal{M}^{l_t}(G, v_t)$, such that

$$G_s^{*l} = \arg \max_{|V_s| \leq k} f(G_s^l) \quad (5)$$

The configuration supports progressive analysis of GNN outputs: (1) For “progressive” diagnosis, answering why $\mathcal{M}$ assigns a wrong label $\mathcal{M}(G, v_t)$ (Example 3), set C as $(G, \mathcal{M}, v_t, \{1, \dots, L\}, L, k)$, where L is the output layer. (2) To analyze $\mathcal{M}$ ’s behavior at selected layers for target layer l_t , set C as $(G, \mathcal{M}, v_t, \mathbb{L}, l_t, k)$. (3) Conventional GNN explanation is a special case with $\mathbb{L} = \{L\}$ and $l_t = L$.

Complexity. We begin hardness analysis by showing that it is feasible to check if a subgraph is an explanation.

LEMMA 1. *Given a configuration $C = (G, \mathcal{M}, v_t, \mathbb{L}, l_t, k)$, and a subgraph G_s of G , it is in PTIME to verify if G_s is an explanation w.r.t. $\mathcal{M}^{l_t}(G, v_t)$ at layer l .*

Proof sketch: We consider a configuration C for a fixed GNN model $\mathcal{M}^{l_t}$ at layer l_t . As shown in cost analyses for GNNs, $\mathcal{M}^{l_t}$ is constant with the same architecture and weights, ensuring deterministic inference in PTIME. To verify G_s , we evaluate a Boolean query for “factual” or “counterfactual” conditions at layer l_t , involving limited calls to $\mathcal{M}$ and equality tests, like $\mathcal{M}(G, v_t) = \mathcal{M}^{l_t}(G_s, v_t)$, all within PTIME.

THEOREM 1. *The decision problem of Explanation generation with model slicing is NP-hard.*

Algorithm 1 : Slice(SS) (single node, single source layer)

Input: a configuration $C = (G, \mathcal{M}, v_t, \{l\}, l_t, k)$;

Output: an explanation G_s^l w.r.t. $\mathcal{M}^{l_t}(G, v_t)$ at GNN layer l .

```

1: initialize  $G_{v_t}^l = (V_{v_t}^l, E_{v_t}^l)$  as the  $l$ -hop subgraph in  $G$ 
2: set  $V_s := \emptyset$ ;  $V_c := \emptyset$ 
3: while  $|V_s| < k$  and  $V_{v_t}^l \setminus V_s \neq \emptyset$  do
4:   expand  $V_s$  by pair-wise node selection in  $V_{v_t}^l$ 
5:   select  $V_c \in V_{v_t}^l$  to keep induced subgraph connected
6:    $V_s' = V_s \cup V_c$ 
7:   set  $G_s^l$  as the subgraph derived from node set  $V_s'$ ;
8:   if Verify( $G_s^l, C$ ) = true then
9:     return  $G_s^l$ ;
10:  while Verify( $G_s^l, C$ ) = false and  $V_{v_t}^l \setminus V_s' \neq \emptyset$  do
11:    replace the lowest-scoring node in  $V_s$  with highest-priority
12:    node in  $V_{v_t}^l \setminus V_s'$  while ensuring connectivity by BFS
13:    set  $G_s^l$  as the subgraph derived from node set  $V_s'$ ;
14:    if Verify( $G_s^l, C$ ) = true then
15:      return  $G_s^l$ ;
16: return  $G_{v_t}^l$ .
```

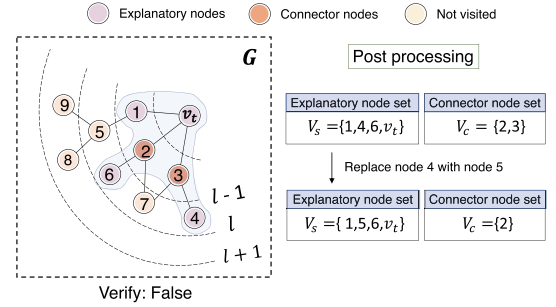


Figure 4: A running example of SliceGX ($k=4$)

Proof sketch: We reduce the maximum coverage problem [28] to our problem by constructing a graph G where nodes represent sets and elements, and edges connect elements to their sets and sets to a target node v_t . A 2-layer GNN $\mathcal{M}$ is trained to label v_t correctly. A size- k factual explanation for v_t exists if and only if there is a solution to the maximum coverage problem with $|\bigcup_{S \in \mathcal{S}} S| \geq b$. Since the decision problem of maximum coverage problem is NP-hard, our problem is also NP-hard.

We next introduce novel and efficient algorithms for layer-wise explanation generation, with guarantees on the quality.

4.4 Generating Model-slicing Explanation

Our first algorithm generates layer-wise explanations for a single node at a designated target layer.

THEOREM 2. *Given a configuration C that specifies a single node v_t and a single source layer l , there is a $\frac{1}{2}$ -approximation for the problem of explanation generation with model-slicing, which takes $O(k|G_{v_t}^l| \log |V_{v_t}^l| + L|V_{v_t}^l|^2)$ time.*

We next introduce such an algorithm as a constructive proof.

Algorithm. The algorithm, denoted as Slice(SS) and illustrated as Algorithm 1, takes a configuration C as input, and generates an

explanation G_s^l for the output $\mathcal{M}^l(G, v_t)$ of a single test node v_t , at a specific layer l .

Initialization (lines 1-2). Algorithm Slice(SS) leverages data locality in GNN inference: For node v_t , an l -hop GNN computes $\mathcal{M}(G, v_t)$ by visiting l -hop neighbors via message passing. Prior work treats the l -hop subgraph as the computational graph, where l is the number of layers [27]. For layer l ($l \in [1, L]$), Slice(SS) initializes $G_{v_t}^l$ as the l -hop subgraph centered at test node v_t , induced by nodes and edges within l -hop of v_t , to refine the search space.

Generation Phase (lines 3-9). Slice(SS) then applies a greedy strategy to select a set of explanatory nodes V_s by iteratively choosing a pair of nodes that maximize a marginal gain of $f(\cdot)$ (line 4). It then induces an explanation with the connect sets by definition (Section 3). Finally it invokes a procedure Verify to check if G_s^l satisfies constraints to be an explanation and return G_s^l if so.

Post-processing (lines 10-15). The generation phase does not guarantee that G_s^l passes Verify. Thus, a lightweight post-processing phase is added. "Back up" nodes are sorted by explainability scores, replacing the lowest-scoring node in V_s while preserving connectivity. The Verify procedure checks if the subgraph yields factual or counterfactual explanations and returns it if valid.

Finally, if no explanation is identified, Slice(SS) returns $G_{v_t}^l$, the l -hop subgraph of v_t , a factual explanation of $\mathcal{M}(G, v_t)$ by default.

Procedure Verify. The procedure verifies if the subgraph G_s^l induced by $V_s \cup V_c$ is a valid explanation, by determining if it is a factual or counterfactual explanation at layer l . Following Lemma 1, the verification is in PTIME.

EXAMPLE 5. Figure 4 illustrates an explanation generated by Slice(SS). The explanatory node set $V_s = \{1, 4, 6, v_t\}$ is initially chosen to maximize the explainability score with the desired size k . To ensure connectivity, connector nodes $V_c = \{2, 3\}$ are added, but the subgraph fails verification. So, node 4 is replaced by node 5, updating V_c to $\{2\}$, resulting in a valid explanation with $V_s = \{1, 5, 6, v_t\}$ and $V_c = \{2\}$.

Correctness and Approximability Algorithm Slice(SS) ensures correctness by selecting V_s and inducing their connect set, such that G_s^l is either a factual or counterfactual explanation at layer l . For approximability, we present the following detailed analysis:

LEMMA 2. Given an output $\mathcal{M}(G, v_t)$ and an explanation G_s^l at layer l , the explainability measure $f(G_s^l)$ is a monotone submodular function for the node set of G_s^l .

The bi-criteria explainability measure $f(\cdot)$ is a submodular function maximizing relative influence $I(V_s^l)$ and embedding diversity $D(V_s^l)$. We then employ the approximation-preserving reduction to a Max-Sum Diversification problem [16] which indicates a greedy algorithm offering a $\frac{1}{2}$ -approximation for this goal. We present the detailed proof in the Appendix B.

Time Cost. Slice(SS) takes $O(|G_{v_t}^l|)$ time to generate l -hop subgraph centered at v_t with breadth-first traversal (line 1). It requires $\lceil \frac{k}{2} \rceil$ iterations of explanatory nodes selection, where each iteration takes $O(T|V_{v_t}^l|)$ to identify the nodes with the maximum marginal gain (based on Equation 3). Here, T is the average time for calculating the marginal gain for each node $v \in V_{v_t}^l$. Next, it requires time

cost in $O(|G_{v_t}^l| * \log |V_{v_t}^l|)$ to induce connect sets and then to induce a connected subgraph to be further verified. Procedure Verify checks whether the subgraph G_s^l is an explanation in $O(l|G_{v_t}^l|)$ time. The post processing takes at most $|V_{v_t}^l|$ rounds and a verification cost in $O(l|V_{v_t}^l|)$ time. Combining these components, as T and $O(l|G_{v_t}^l|)$ are relatively small compared to the dominant terms, the overall time cost is in $O(k|G_{v_t}^l| \log |V_{v_t}^l| + l|V_{v_t}^l|^2)$. Here, $l|V_{v_t}^l|^2$ comes from the post-processing stage (line 10-15, Alg 1). Empirically, such worst-case scenarios occur in only about 3% of instances, so the actual cost is much lower in practice.

With the above analysis, Theorem 2 follows.

As Slice(SS) method generates explanations sequentially for each test node, it is computationally expensive, especially when dealing with multiple test nodes and layers. To address this limitation, we introduce two optimization techniques, Slice(MS) and Slice(MM), to incrementally generate explanations. Specifically, Slice(MS) updates auxiliary structures for all test nodes V_T during the node selection phase. This optimization reduces redundant computations and enhances scalability for large-scale scenarios. Building on Slice(MS), Slice(MM) further extends the approach to handle multi-layer scenarios by introducing a "hop jumping" strategy that tracks explanatory node sets across multiple layers. This enables Slice(MM) to identify explanations that are both efficient and comprehensive across different layers of the network. Due to the limited space, we provide the details in the Appendix C.

5 Experimental Study

To evaluate SliceGX-based explanation algorithms, We investigate the following questions. **RQ1:** Can SliceGX-based algorithms generate high-quality explanations for designated layers? **RQ2:** What are the impact of important factors to the quality and efficiency of explanation generation? **RQ3:** How fast can SliceGX generate explanations in a progressive manner? We also illustrate SliceGX-based explanation for model debugging. **Our code is given at [1].**

5.1 Experimental Setup

Datasets. We adopt six benchmark graph datasets, as summarized in Table 2: **BA-shapes (BA)** [14], **Tree-Cycles (TR)** [14], **Cora (CO)** [25], **Coauthor CS (CS)** [35], **YelpChi (YC)** [12], and **Amazon (AM)** [45]. These datasets cover synthetic and real-world graphs for tasks such as node classification, spam detection, and co-authorship analysis. We provide the details in the Appendix D.

GNN classifiers. For all datasets, we have trained 3-layer Graph Convolutional Networks (GCNs) [25] for comparison. We used the Adam optimizer [11] with a learning rate of 0.001 for 2000 epochs. We also showcase that SliceGX can be applied to other types of GNNs in the Appendix E.

GNN explainers. We have implemented the following explainers: (1) our SliceGX method based on Slice(SS). (2) GNNExplainer [43] uses mutual information to identify key edges and node features for instance-level classification. (3) SubgraphX [44] applies Monte Carlo tree search to find important subgraphs via node pruning for graph classification. (4) PGExplainer [27] models edges as conditionally independent Bernoulli variables, optimized for mutual

Table 1: Fidelity Evaluation of GNN Explainers. Fidelity+ (Fid+, higher is better $\uparrow$) and Fidelity- (Fid-, lower is better $\downarrow$). The best and second-best performances are denoted in bold and underlined fonts, respectively. ‘—’ indicates that the method failed to complete within the 24-hour time limit.

Explainers	BA-shapes		Tree-Cycles		Cora		Coauthor CS		YelpChi		Amazon	
	Fid+ ($\uparrow$)	Fid- ($\downarrow$)	Fid+ ($\uparrow$)	Fid- ($\downarrow$)	Fid+ ($\uparrow$)	Fid- ($\downarrow$)	Fid+ ($\uparrow$)	Fid- ($\downarrow$)	Fid+ ($\uparrow$)	Fid- ($\downarrow$)	Fid+ ($\uparrow$)	Fid- ($\downarrow$)
GNNExplainer	0.2665	0.7264	0.6065	0.6611	0.1088	0.8173	0.4302	0.1525	0.2041	0.4238	—	—
PGExplainer	0.2763	0.6697	0.3689	0.2551	0.2739	0.2041	0.3217	0.2815	<u>0.3482</u>	0.4255	<u>0.0421</u>	0.8469
GraphMask	0.4302	<u>0.3580</u>	0.1243	<u>0.0280</u>	0.1260	0.7202	0.2341	0.4512	<u>0.1245</u>	0.7241	0.0012	<u>0.7187</u>
SubgraphX	0.2307	0.4018	<u>0.7012</u>	0.0782	0.4748	<u>0.1003</u>	0.5142	0.1451	—	—	—	—
SAME	0.0303	0.6662	0.5124	0.4920	0.3984	0.2522	0.3516	<u>0.1242</u>	—	—	—	—
FlowX	<u>0.4682</u>	0.7062	0.5052	0.2583	<u>0.6430</u>	0.7471	<u>0.5189</u>	0.1432	0.3014	<u>0.2414</u>	—	—
SliceGX	0.6918	0.0670	0.8047	0	0.7117	0.0531	0.7003	0.0341	0.5014	0.1294	0.1724	0.4521

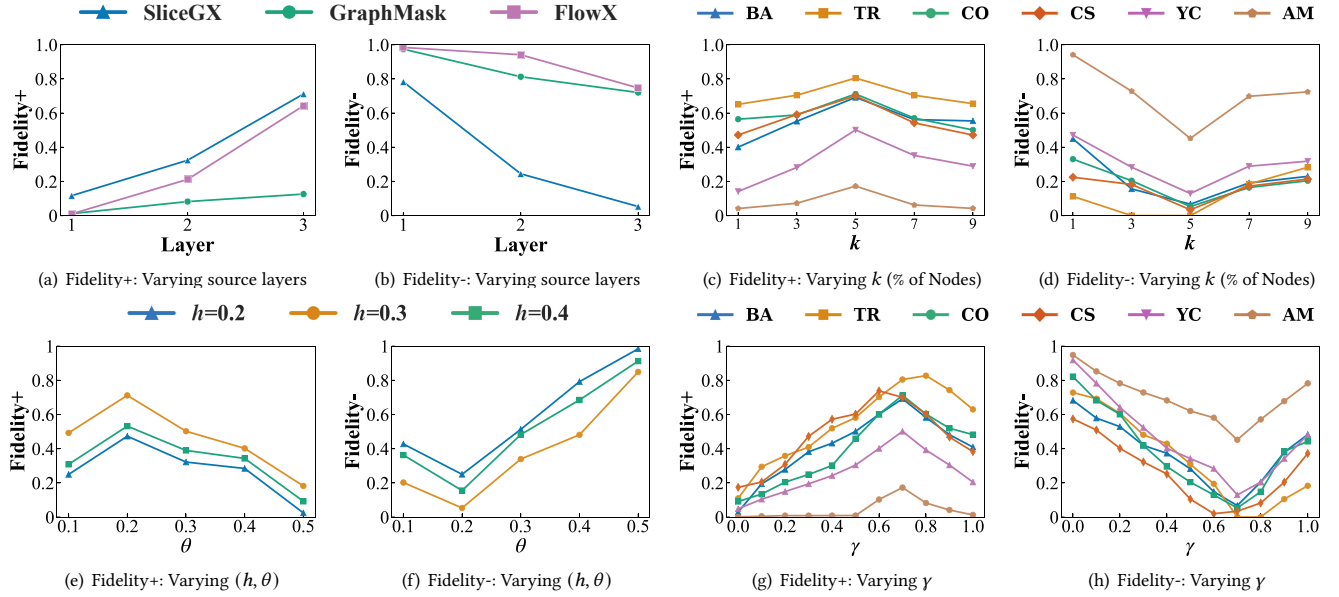


Figure 5: Impact of factors on quality of explanations

information. (5) GraphMask [33] trains edge masks to prune irrelevant edges and generate layer-specific subgraphs. (6) SAME [42] uses two-phase expansion MCTS to efficiently find optimal multipiece explanations via k -hop Shapley. (7) FlowX [17] uses a sampling scheme to compute Shapley values, which quantify the importance of “message flows” to the model’s prediction. We also provide the results for Slice(MS) and Slice(MM) in the Appendix E.

Evaluation metrics. We extend Fidelity [23, 30] for layer-wise explanations. It measures faithfulness of explanations at layer l with the normalized difference between the output of a GNN over G and $G \setminus G_s^l$, where G_s^l is the explanation at layer l .

(1) We define **Fidelity+** as:

$$Fidelity+ = \frac{1}{|V_T|} \sum_{v_t \in V_T} (\phi^l(G, v_t)_c - \phi^l(G \setminus G_s^l, v_t)_c) \quad (6)$$

where $\phi^l(\cdot)_c$ is the predicted likelihood of label c for v_t from M^l , and c is the label assigned by $M^l(G, v_t)$. Fidelity+ quantifies the impact of removing the explanation on label assignment, with higher values indicating better explanations.

(2) Similarly, we define **Fidelity-** of G_s^l as:

$$Fidelity- = \frac{1}{|V_T|} \sum_{v_t \in V_T} (\phi^l(G, v_t)_c - \phi^l(G_s^l, v_t)_c) \quad (7)$$

Fidelity- at layer l quantifies how “consistent” the result of GNN is over G_s^l compared with its result at layer l over G . The smaller the value is, the better.

We also report the time cost of explanation generation. For learning-based explainer, the cost includes learning overhead.

5.2 Experimental Results

Exp-1: Quality of explanations. We evaluated explanation quality for target layer output, setting source layer $L = \{3\}$, target layer $l_t = 3$. The evaluation was conducted on a test set of 100 sampled, labeled nodes ($|V_T| = 100$), with the explanation size k set to 5% of the total nodes in each graph.

Overall Fidelity. We conducted a quantitative fidelity evaluation, with the results summarized in Table 1. Our method, SliceGX, consistently outperforms baseline explainer across all datasets, achieving the best performance in both Fidelity+ and Fidelity- metrics.

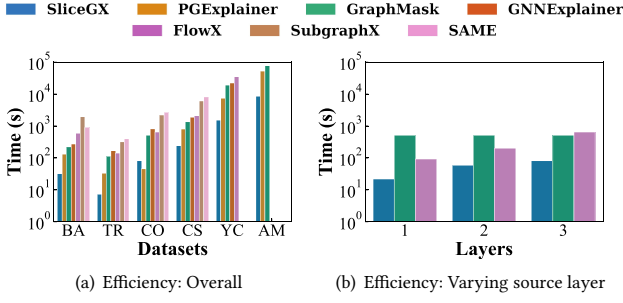


Figure 6: Efficiency of explanation generation

Notably, SliceGX obtains a Fidelity- score of 0 on the Tree-Cycles dataset—a theoretically optimal result. This demonstrates a substantial improvement over existing state-of-the-art GNN explainers.

Impact of source layers. We varied source layers from 1 to 3 (Figs. 5(a), 5(b)) to assess robustness with a fixed target layer. Here, we selected GraphMask and FlowX as the baseline explainers for comparison, as other existing explainers lack support for intermediate layer explanations. For both Fidelity+ and Fidelity-, SliceGX consistently outperforms both GraphMask and FlowX across all layers. This demonstrates SliceGX’s superior ability to generate robust and high-quality layer-wise explanations.

Impact of k . The variation of k from 1% to 9% of nodes in each graph was employed in the analysis. Shown in Figs. 5(c) and 5(d), the best fidelity performance across all datasets was achieved at $k=5\%$. Thus we selected $k=5\%$ as the optimal configuration, as it provides the strongest explainer validity while maintaining sparsity.

Impact of (h, θ) . We evaluated thresholds h (influence) and θ (embedding diversity) on the Cora dataset. Figs. 5(e) and 5(f) show: (1) Proper embedding diversity ($\theta \in [0.2, 0.3]$) improves explanations by mitigating bias from similar nodes, but excessive diversity introduces noise and reduces quality. (2) Higher influence ($h \in [0.3, 0.4]$) improves explanations, but overly influential nodes can introduce bias if their embeddings differ significantly from target nodes.

Impact of γ . We also evaluated the balance factor γ between relative influence and embedding diversity in Equation 3 across all datasets. As shown in Figs. 5(g) and 5(h), when γ is too small (i.e., $\gamma < 0.3$), the objective favors embedding diversity, potentially admitting noisy nodes and degrading explanation quality. Conversely, when γ is too large (i.e., $\gamma > 0.8$), The objective overemphasizes influence, biasing explanations toward influential yet redundant nodes. SliceGX achieves the best performance when γ lies in the moderate range (i.e., $\gamma \in [0.6, 0.8]$), effectively balancing influence and diversity to yield high-quality explanations. These results validate our bi-criteria objective, demonstrating that SliceGX adapts to diverse explanation needs by tuning γ .

Exp-2: Efficiency of explanation generation. We next report the efficiency of SliceGX, compared with other explainers.

Overall performance. Figure 6(a) details the time costs (under settings from Table 1), SliceGX is by far the most efficient GNN explainer. It is up to 6 to 55 times faster than other competitors. This efficiency is critical, as SAME and SubgraphX incur high overhead

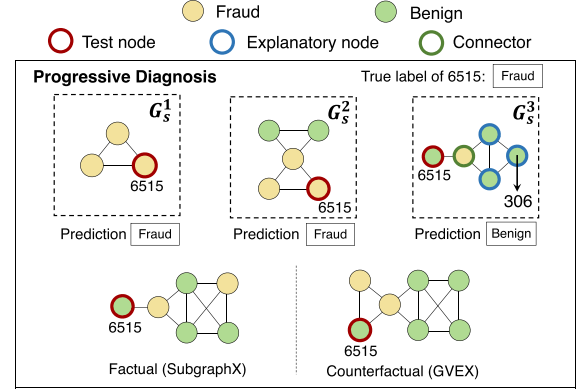


Figure 7: Case study on progressive diagnose mode

from costly Shapley value computations and tree search algorithms, respectively, and fail to complete on YC and AM (after 24 hours). Ultimately, only SliceGX, PGExplainer, and GraphMask were efficient enough to run successfully on the large AM dataset.

Impact of layers. As shown in Figure 6(b), SliceGX runs faster when lower source layers, confirming its reliance on data locality and l -hop neighbor exploration. Its progressive generation facilitates early stopping for pertinent explanations, enhancing efficiency. In contrast, GraphMask remains insensitive to the choice of l as it simultaneously learns masks for all layers.

Exp-3: Case study. We next showcase the application of SliceGX for diagnose mode. As shown in Figure 7, assume a user aims to find “why” a GCN $\mathcal{M}$ misclassified node 6515 (with true label “Fraud”) as prediction “Benign” and “when” (at which layer) it fails (Figure 7). Using SliceGX in “progressive diagnosis” mode, explanations G_s^1 , G_s^2 , and G_s^3 reveal that $\mathcal{M}$ remains correct up to the second layer. Comparing G_s^2 and G_s^3 , the error likely stems from the node 306 (“Benign”) at hop 3, leading to a failure at layer 3. For model debugging, this helps fine-tune layer 3 or switching to a 2-layer GNN to improve accuracy. SliceGX can provide finer-grained, node- and layer-level explanations, whereas SubgraphX [44] and GVEX [9] – two state-of-the-art GNN explainability methods – only generate a singular, final output explanation. This limitation hinders error tracing through layers, complicating misclassification diagnosis.

6 Conclusion

We introduced SliceGX, a layer-wise, novel GNN explanation framework that leverages model slicing to generate explanations from selected source layers for output at target layers. We formalized explanation generation as a novel bi-criteria optimization problem, and introduced novel and efficient algorithms for both single-node, single-source and multi-node, multi-source settings. Our theoretical and experimental results demonstrate the effectiveness, efficiency, and scalability of SliceGX algorithms, as well as their practical utility for model debugging in real-world applications.

Acknowledgments

This work was supported by NSFC Grant No. 62502434 and the Ningbo Yongjiang Talent Introduction Programme (2022A-237-G).

References

- [1] 2025. Code. <https://github.com/jinzhaonet/SliceGX>.
- [2] Julius Adebayo, Michael Muell, Ilaria Liccardi, and Been Kim. 2020. Debugging tests for model explanations. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [3] Marcelo Arenas. 2024. A data management approach to explainable AI. In *Companion of the 43rd Symposium on Principles of Database Systems*. 1–3.
- [4] Mohit Bajaj, Lingyang Chu, Zi Yu Xue, Jian Pei, Lanjun Wang, Peter Cho-Ho Lam, and Yong Zhang. 2021. Robust counterfactual explanations on graph neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 5644–5655.
- [5] Arnaud Blouin, Benoît Combemale, Benoît Baudry, and Olivier Beaudoux. 2011. Modeling model slicers. In *International Conference on Model Driven Engineering Languages and Systems*. 62–76.
- [6] Shaofeng Cai, Gang Chen, Beng Chin Ooi, and Jinyang Gao. 2019. Model slicing for supporting complex analytics with elastic inference cost and resource constraints. *Proc. VLDB Endow.* 13, 2 (2019), 86–99.
- [7] Mengru Chen, Chao Huang, Lianghao Xia, Wei Wei, Yong Xu, and Ronghua Luo. 2023. Heterogeneous graph contrastive learning for recommendation. In *Proceedings of the sixteenth ACM international conference on web search and data mining*. 544–552.
- [8] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. 2020. Scalable graph neural networks via bidirectional propagation. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Tingyang Chen, Dazhuo Qiu, Yinghui Wu, Arijit Khan, Xiangyu Ke, and Yunjun Gao. 2024. View-based explanations for graph neural networks. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27.
- [10] Yu Christine Chen, Jianhui Wang, Alejandro D Domínguez-García, and Peter W Sauer. 2015. Measurement-based estimation of the power flow Jacobian matrix. *IEEE Transactions on Smart Grid* 7, 5 (2015), 2507–2515.
- [11] P Kingma Diederik. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.
- [12] Yingdong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S. Yu. 2020. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 315–324.
- [13] Alexandre Duval and Fragkiskos D Malliaros. 2021. Graphsvx: Shapley value explanations for graph neural networks. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. 302–318.
- [14] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [15] Thorben Funke, Megha Khosla, Mandeep Rathee, and Avishek Anand. 2022. Zorro: Valid, sparse, and stable explanations in graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 8687–8698.
- [16] Sreenivas Gollapudi and Aneesh Sharma. 2009. An axiomatic approach for result diversification. In *International conference on World Wide Web (WWW)*. 381–390.
- [17] Shurui Gui, Hao Yuan, Jie Wang, Qicheng Lao, Kang Li, and Shuiwang Ji. 2023. Flowx: Towards explainable graph neural networks via message flows. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 7 (2023), 4567–4578.
- [18] Refael Hassin, Shlomi Rubinstein, and Arie Tamir. 1997. Approximation algorithms for maximum dispersion. *Operations research letters* 21, 3 (1997), 133–137.
- [19] Ke-Jou Hsu, Ketan Bhardwaj, and Ada Gavrilovska. 2019. Couper: DNN model slicing for visual analytics containers at the edge. In *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing (SEC)*. 179–194.
- [20] Qiang Huang, Makoto Yamada, Yuan Tian, Dinesh Singh, and Yi Chang. 2022. Graphlime: Local interpretable model explanations for graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2022), 6968–6972.
- [21] Zhenhua Huang, Wenhao Zhou, Yufeng Li, Xiuyang Wu, Chengpei Xu, Junfeng Fang, Zhaohong Jia, Linyuan Lü, and Feng Xia. 2025. SEHG: Bridging Interpretability and Prediction in Self-Explainable Heterogeneous Graph Neural Networks. In *Proceedings of the ACM on Web Conference 2025*. 1292–1304.
- [22] Md Tauhid Bin Iqbal, Abdul Muqet, and Sung-Ho Bae. 2022. Visual interpretation of CNN prediction through layerwise sequential selection of discernible neurons. *IEEE Access* 10 (2022), 81988–82002.
- [23] Jaykumar Kakkad, Jaspal Jannu, Kartik Sharma, Charu Aggarwal, and Sourav Medya. 2023. A survey on explainability of graph neural networks. *IEEE Data Eng. Bull.* 46, 2 (2023), 35–63.
- [24] Samir Khuller, Anna Moss, and Joseph Seffi Naor. 1999. The budgeted maximum coverage problem. *Information processing letters* 70, 1 (1999), 39–45.
- [25] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.
- [26] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.
- [27] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. In *Advances in neural information processing systems (NeurIPS)*.
- [28] Christos Papadimitriou and Mihalis Yannakakis. 1988. Optimization, approximation, and complexity classes. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*. 229–234.
- [29] Phillip E Pope, Soheil Kolouri, Mohammad Rostami, Charles E Martin, and Heiko Hoffmann. 2019. Explainability methods for graph convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10772–10781.
- [30] Mario Alfonso Prado-Romero, Bardh Prenkaj, Giovanni Stilo, and Fosca Giannotti. 2024. A survey on graph counterfactual explanations: definitions, methods, evaluation, and research challenges. *Comput. Surveys* 56, 7 (2024), 1–37.
- [31] Tilman Räuher, Anson Ho, Stephen Casper, and Dylan Hadfield-Menell. 2023. Toward transparent AI: A survey on interpreting the inner structures of deep neural networks. In *Secure and Trustworthy Machine Learning (SaTML)*. 464–483.
- [32] Aravind Sankar, Yozen Liu, Jun Yu, and Neil Shah. 2021. Graph neural networks for friend ranking in large-scale social platforms. In *Proceedings of the Web Conference 2021*. 2535–2546.
- [33] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. 2021. Interpreting graph neural networks for NLP with differentiable edge masking. In *International Conference on Learning Representations (ICLR)*.
- [34] Thomas Schnake, Oliver Eberle, Jonas Lederer, Shinichi Nakajima, Kristof T Schütt, Klaus-Robert Müller, and Grégoire Montavon. 2021. Higher-order explanations of graph neural networks via relevant walks. *IEEE transactions on pattern analysis and machine intelligence* 44, 11 (2021), 7581–7596.
- [35] Oleksandr Shchur, Maximilian Mummé, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [36] Ihsan Ullah, Andre Rios, Vaibhav Gala, and Susan McKeever. 2021. Explaining deep learning models for tabular data using layer-wise relevance propagation. *Applied Sciences* 12, 1 (2021), 136.
- [37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph attention networks. In *International Conference on Learning Representations (ICLR)*.
- [38] Baowen Xu, Ju Qian, Xiaofang Zhang, Zhongqiang Wu, and Lin Chen. 2005. A brief survey of program slicing. *ACM SIGSOFT Software Engineering Notes* 30, 2 (2005), 1–36.
- [39] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks?. In *International Conference on Learning Representations (ICLR)*.
- [40] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. In *International conference on machine learning*. PMLR, 5453–5462.
- [41] Han Yang, Junyuan Fang, Jiajing Wu, Dan Li, Yaonan Wang, and Zibin Zheng. 2025. Soft label enhanced graph neural network under heterophily. *Knowledge-Based Systems* 309 (2025), 112861.
- [42] Ziyuan Ye, Rihan Huang, Qilin Wu, and Quanying Liu. 2023. SAME: Uncovering GNN black box with structure-aware shapley-based multipiece explanations. *Advances in Neural Information Processing Systems* 36 (2023), 6442–6466.
- [43] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GnnExplainer: Generating explanations for graph neural networks. In *Advances in neural information processing systems (NeurIPS)*. 9240–9251.
- [44] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *International conference on machine learning*. 12241–12252.
- [45] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2020. Graphsaint: Graph sampling based inductive learning method. In *International Conference on Learning Representations (ICLR)*.
- [46] Lingze Zeng, Naili Xing, Shaofeng Cai, Gang Chen, Beng Chin Ooi, Jian Pei, and Yuncheng Wu. 2025. Powering in-database dynamic model slicing for structured data analytics. *Proc. VLDB Endow.* (2025).
- [47] Wei Zhang, Xiaofan Li, and Wolfgang Nejdl. 2024. Adversarial mask explainer for graph neural networks. In *Proceedings of the ACM Web Conference 2024*. 861–869.
- [48] Wentao Zhang, Zhi Yang, Yexin Wang, Yu Shen, Yang Li, Liang Wang, and Bin Cui. 2021. Grain: Improving data efficiency of graph neural networks via diversified influence maximization. *Proc. VLDB Endow.* 14, 11 (2021), 2473–2482.
- [49] Xingyi Zhang, Jinchao Huang, Fangyuan Zhang, and Sibao Wang. 2024. FICOM: an effective and scalable active learning framework for GNNs on semi-supervised node classification. *The VLDB Journal* 33, 5 (2024), 1–20.
- [50] Yongqi Zhang, Zhanke Zhou, Quanming Yao, Xiaowen Chu, and Bo Han. 2023. Adaprop: Learning adaptive propagation for graph neural network based knowledge graph reasoning. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 3446–3457.
- [51] Ziqi Zhang, Yuanchun Li, Yao Guo, Xiangqun Chen, and Yunxin Liu. 2020. Dynamic slicing for deep neural networks. In *ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 838–850.

A Discussion

In this work, we propose a sliced model, which is derived from a pre-trained GNN model. As described in Section 4.1, an l -sliced model $\mathcal{M}^l$ consists of two primary components: the feature extractor (denoted as f_1^l) and the predictor (denoted as f_2). In practice, for an l -sliced model of a pre-trained GNN $\mathcal{M}$ with L layers, the f_1^l component is constructed by taking the first l pre-trained GNN layers to capture hidden features from the graph data. Following the f_1^l component, it connects the original MLP (Multi-Layer Perceptron) layers of the GNN, constituting the f_2 part of our l -sliced model.

There are three options to construct an l -sliced model:

Option 1: Retrain both f_1^l and f_2 ;

Option 2: Freeze pre-trained f_1^l and retrain f_2 ; and

Option 3: Directly use pre-trained f_1^l and f_2 (adopted by SliceGX).

Compared to the first two options, we favor option 3 in this paper for three reasons: (a) The original model $\mathcal{M}$ can be treated as a black box as we focus solely on integrating layers without knowledge of internal workings of $\mathcal{M}$; (b) it saves time when directly utilizing the pre-trained components instead of engaging in time-consuming retraining processes; and (c) even if choosing option 1 or option 2 to retrain the model, inconsistencies in the final outputs could still exist.

As option 3 also generates a “new” model $\mathcal{M}^l$ with potential inconsistencies, we introduce *guard condition* for explanation at layer l , including “factual” and “counterfactual,” to mitigate this issue (Section 4.1). For instance, by ensuring that the prediction of a test node v_t using an explanation subgraph G_s^l in the sliced model $\mathcal{M}^l$ is consistent with the prediction of v_t using original graph G in both $\mathcal{M}$ and $\mathcal{M}^l$, the explanation generated by option 3 remains “faithful” to the original model, even without any retraining. This is the rationale behind the necessity of the guard condition. However, as both Option 1 and Option 2 are more likely to violate the guard condition, it becomes difficult to generate high-quality explanations that stays faithful to the original model’s behavior. The main reasons are as follows: Although option 1 enables the sliced model $\mathcal{M}^l$ to fit the original data better, it poses a substantial risk of altering the original model’s behavior. As our primary objective is to generate explanations based on the original GNN model, retraining both components would create a “new” model that may not capture the characteristics of the original architecture and potentially change the inference process. Similarly, option 2 might make the new sliced model $\mathcal{M}^l$ achieve closer accuracy to the final output of the original GNN, but it still introduces inconsistencies in model inference.

Based on derived sliced models, the design of SliceGX is inherently grounded in both the predictions of the original model and the sliced model in terms of guard condition, rather than treating the sliced model as an independent entity. However, for existing methods, such as GNNExplainer [43] and SubgraphX [44], even when applying directly to the sliced model, their explanations are not high-quality to provide truly progressive explanations that are faithful to both original model and sliced model, as they neglect the critical logical connection between the sliced model and the original model and are only accountable to the sliced model.

B Proofs

B.1 Proof of Theorem 1. *The decision problem of Explanation generation with model slicing is NP-hard.*

Proof: We perform a reduction from the maximum coverage problem [24]. Given a collection of sets $\mathcal{S} = S_1, \dots, S_n$, and an integer k , the maximum coverage problem is to find a subset $\mathcal{S}' \subseteq \mathcal{S}$ such that $|\mathcal{S}'| \leq k$, and the number of elements $|\bigcup_{S \in \mathcal{S}'} S|$ is maximized (or $\geq b$ for a given threshold b , for its decision version).

Given k , $\mathcal{S}'$, and $\mathcal{S}$, we construct a configuration as follows. (1) We construct a graph G with $V = \{v_t\} \cup V_1 \cup V_2$, where v_t is a designated target node; for each set $S_i \in \mathcal{S}$, there is a node v_{S_i} in V_1 ; and for each element $x_j \in S_i \in \mathcal{S}$, there is a distinct node v_j in V_2 . (2) For every element $x_j \in S_i$, there is an edge (v_j, v_{S_i}) in E . For every node v_{S_i} , there is an edge between v_{S_i} and v_t . (3) Assign all the nodes in G a same ground truth label. Train a 2-layer vanilla GNN $\mathcal{M}$ that assigns a correct label to v_t (which is in PTIME [8]). As $\mathcal{M}$ is a fixed model that takes as the same input G as both training and test data, the inference ensures the invariance property of GNNs and assigns $\mathcal{M}(G, v)$ with ground truth label for every $v \in V$. For each node v_{S_i} , the relative influence set $c(v_{S_i})$ is exactly the set of nodes $\{v_{ij}\}$, which corresponds to the set S_i and its elements. Then there exists a size k factual explanation of v_t with explainability at least b , if and only if there is a solution for maximum coverage with $|\bigcup_{S \in \mathcal{S}'} S| \geq b$. As the maximum coverage problem is known to be NP-hard [24], our problem is NP-hard.

B.2 Proof of Lemma 2. *Given an output $\mathcal{M}(G, v)$ to be explained, and an explanation G_s^l at layer l , the explainability measure $f(G_s^l)$ is a monotone submodular function for the node set of G_s^l .*

Proof: As $f(G_s^l)$ is the sum of two node set functions $I(V_s^l)$ and $D(V_s^l)$ defined in Equation 3, we prove that the two functions are both monotone submodular. We first analyze the property of $I(V_s^l)$. (1) It is clear that the relative influence set $c(v)$ for any $v \in V_s^l$ is non-negative. So $I(V_s^l)$ is non-decreasing monotone function. (2) Next we will show that for any set $V_{s'} \subset V_s$ and any node $u \notin V_s$, $I(\cdot)$ is submodular by verifying the following function:

$$I(V_{s'} \cup \{u\}) - I(V_{s'}) \geq I(V_s \cup \{u\}) - I(V_s)$$

If $c(\{u\}) \cap c(V_s) = \emptyset$, we have $I(V_s \cup \{u\}) = I(V_s) + I(u)$ and $I(V_{s'} \cup \{u\}) = I(V_{s'}) + I(u)$, which satisfies the equation. **Case (ii).** Otherwise, we assume that $I(V_{s'} \cup \{u\}) = I(V_{s'}) + \Delta I(u|V_{s'})$ and $I(V_s \cup \{u\}) = I(V_s) + \Delta I(u|V_s)$, where $\Delta I(\cdot)$ indicates the marginal gain by adding the new element u . Due to the non-negative nature of $c(V_s \setminus V_{s'})$, we have $|c(u) \cap c(V_s \setminus V_{s'})| \geq 0$, which indicates that $\Delta I(u|V_{s'}) \geq \Delta I(u|V_s)$, thus satisfying the inequality.

Following a similar analysis, we can show that $D(\cdot)$ is also monotone submodular. To prove that, we need to show that the function satisfies the diminishing returns property. Considering the same condition $V_{s'} \subset V_s$ and any node $u \notin V_s$, $D(V_{s'} \cup \{u\}) - D(V_{s'}) \geq D(V_s \cup \{u\}) - D(V_s)$. According to union property, since $V_{s'} \subset V_s$, the contribution of $r(u)$ to the union $\bigcup_{v \in V_{s'} \cup \{u\}} r(v)$ will be greater than or equal to its contribution to $\bigcup_{v \in V_s \cup \{u\}} r(v)$ due to overlapping elements in $r(u)$ that are already covered by V_s . This means that the increase in $\left| \bigcup_{v \in V_{s'} \cup \{u\}} r(v) \right|$ from adding u to $V_{s'}$ is at least as large as the increase in $\left| \bigcup_{v \in V_s \cup \{u\}} r(v) \right|$ from adding u to V_s .

Algorithm 2 : Slice(MS) (multi nodes, single source layer)

Input: a configuration $C = (G, \mathcal{M}, V_T, \{l\}, l_t, k)$;
Output: a set of explanations $\mathcal{G}_s^l$ for each $v_t \in V_T$ w.r.t. $\mathcal{M}^{l_t}(G, v_t)$ at GNN layer l .

- 1: $G^l(V_T) = \bigcup_{v_t \in V_T} G^l(v_t)$ // union l -hop of V_T
- 2: Set $\mathcal{V}_s[i] = \emptyset$ for $i \in [0, |V_T| - 1]$
- 3: **for** $i \in [0, |V_T| - 1]$ **do**
- 4: **for** $v \in G^l(v_t)$, where v_t is the i -th element in V_T , **do**
- 5: set $v.B[i] = \text{true}$ // a boolean indicator
- 6: set $V'_s = G^l(V_T)$
- 7: **for** $i \in [0, |V_T| - 1]$ **do**
- 8: **while** $|\mathcal{V}_s[i]| < k$ and $V'_s \neq \emptyset$ **do**
- 9: select $\{v_1^*, v_2^*\} \in V'_s$ using pair-wise node selection in Slice(SS) (line 4) based on i -th element in V_T
- 10: **for** $j \in [0, |V_T| - 1]$ **do**
- 11: **if** $v_1^*.B[j] = \text{true}$ **then**
- 12: update $\mathcal{V}_s[j] = \mathcal{V}_s[j] \cup \{v_1^*\}$
- 13: **if** $v_2^*.B[j] = \text{true}$ **then**
- 14: update $\mathcal{V}_s[j] = \mathcal{V}_s[j] \cup \{v_2^*\}$
- 15: IncVerify(MS) ($\mathcal{V}_s[i], C$)
- 16: $V'_s = V'_s \setminus \{v_1^*, v_2^*\}$
- 17: **return** $\mathcal{G}_s^l$.

Algorithm 3 : Slice(MM) (multi nodes, multi source layers)

Input: a configuration $C = (G, \mathcal{M}, V_T, \mathcal{L}, l_t, k)$;
Output: a set of explanations $\mathcal{G}_s = \{\mathcal{G}_s^1, \dots, \mathcal{G}_s^L\}$ for each $v_t \in V_T$ and layer $l \in \mathcal{L}$ w.r.t. $\mathcal{M}^{l_t}(G, v)$ at GNN layer l .

- 1: $G^L(V_T) = \bigcup_{v_t \in V_T} G^L(v_t)$
- 2: $\forall v_{t_i} \in V_T, \forall l_j \in \mathcal{L}$, set $\mathcal{V}_B[l_i][j] = \emptyset$
- 3: sort $\mathcal{L}$ in a decreasing order
- 4: **for** $l_j \in \mathcal{L}$ **do**
- 5: invoke Slice(MS) to update $\mathcal{V}_B[l_i][j]$ w.r.t. layer l_j
- 6: set $\mathcal{G}_s^{l_j}$ as a set of subgraphs derived from $\mathcal{V}_B[l_i][j]$
- 7: **for** $v_{t_i} \in V_T$ **do**
- 8: **if** v_{t_i} has no explanation at layer l_j **and** $l_j \neq \mathcal{L}$ **then**
- 9: $\mathcal{V}_B[l_i][j+1] = \mathcal{V}_B[l_i][j] \setminus \{v | v \notin N^{l_{j+1}}(v_{t_i})\}$
- 10: **return** $\mathcal{G}_s$.

satisfying the diminishing returns property required for submodularity. Putting these together, the bi-criteria explainability measure $f(G_s^l)$ is a monotone submodular function.

B.3 Proof of Theorem 2. Given a configuration C , with a specific test node v and a specific GNN layer l , there is a $\frac{1}{2}$ -approximate algorithm for generating model-slicing explanation.

Proof: Given a configuration C , with a specific test node v and a specific GNN layer l , we reformulate the problem as node selection problem, analogous to a counterpart as a Max-Sum Diversification task, with a modified facility dispersion objective $f'(V_s) = \gamma \sum_{v \in V_s^l} |c(v)| + (1 - \gamma) \sum_{u, v \in V_s^l} \mathbf{1}[d(Z_u^l, Z_v^l) \geq \theta]$. Thus, given the approximation preserving reduction, we can map known results about Max-Sum Diversification to the diversification objective. We observe that maximizing the objective is NP-Hard, but there are known approximation algorithms for the problem [18], which yields a $\frac{1}{2}$ -approximation.

C SliceMS and SliceMM

C.1 Multi-nodes, Single-Source Layers. The decision problem of Explanation generation with model slicing is NP-hard even for a single source layer ($|\mathcal{L}| = 1$). In Slice(SS), the explanations are generated for each test node sequentially, which is inefficient. To address this limitation, Slice(MS) employs an incremental approach, simultaneously selecting explanatory nodes for all test nodes V_T during Generation Phase in Algorithm 1.

Algorithm. The algorithm, denoted as Slice(MS) and illustrated as Algorithm 2.

Initialization (lines 1-6). Algorithm 2 starts with setting $G^l(V_T)$ as all potential explanatory nodes for V_T because of data locality of GNN inference. Next, it introduces entry $\mathcal{V}_s[i]$ to store the “current” explanatory node set of the i -th element in V_T , and a boolean value $v.B[i]$ to represent whether a potential explanatory node $v \in G^l(V_T)$ is likely to join $\mathcal{V}_s[i]$. For i -th element in V_T , denoted as v_{t_i} , $\mathcal{V}_s[i]$ is initialized to an empty set, indicating that no explanatory nodes have been selected for v_{t_i} , and each node $v \in G^l(v_{t_i})$ is a “candidate” explanatory node for v_{t_i} , i.e., $v.B[i] = \text{true}$. In addition, it maintains a global candidate explanatory node set V'_s (line 6).

Incremental generation phase (lines 7-17). Slice(MS) follows a pair-wise node selection based on a greedy strategy similar to Slice(SS). When a new node $v \in \{v_1^*, v_2^*\}$ is introduced for maximum marginal gain corresponding to at least one test node, it uses v to update explanatory node sets $\mathcal{V}_s[i]$. Specifically, if $v.B[i]$ is true, it indicates that v is a “promising” explanatory node for the i -th test node. Thus, the explanatory sets of such test nodes influenced by v can be updated all at once. When an explanatory node set of a test node in $\mathcal{V}_s[i]$ is updated, it generates a subgraph G_s with connectivity. Next, it invokes Procedure IncVerify(MS) to test if G_s is an explanation for multiple test nodes in V_T .

Procedure IncVerify(MS). The procedure verifies if the subgraph G_s induced by explanatory node set V_s serves as an explanation for multiple test nodes in V_T . Specifically, for all node $v \in V_s$, if $v.B[i]$ is true, which means that V_s can serve as a candidate explanation for the i -th test node v_{t_i} , then it invokes Verify(SS) to verify if G_s is an explanation for test nodes that satisfy the verification condition in Section 4.1. If so, it updates $\mathcal{V}_s[i]$ as the nodes in G_s and reduces V_T to retain only those that have no explanation to further explore explanations for them in next loop.

C.2 Multi-nodes, Multi-Source Layers. We describe the Slice(MM) procedure, illustrated in Algorithm 3. Slice(MM) processes the layers $\mathcal{L}$ sequentially, starting from the last layer and moving downward (i.e., layer 1). For each layer, it performs two main steps: (1) It executes a “hop jumping” process that prunes the explanatory node set by removing nodes beyond l hops when at layer l . This layer-by-layer updating and pruning enhance efficiency by narrowing the search space to relevant nodes, thus avoiding unnecessary computations on irrelevant nodes for the current layer’s output. (2) It updates a map $\mathcal{V}_B$ using Slice(MS), where each entry $\mathcal{V}_B[l_i][j]$ contains the best explanatory node set $V_s^{l_j}$ for the output $\mathcal{M}^{l_t}(G, v_{t_i})$ concerning $v_{t_i} \in V_T$ at layer l_j .

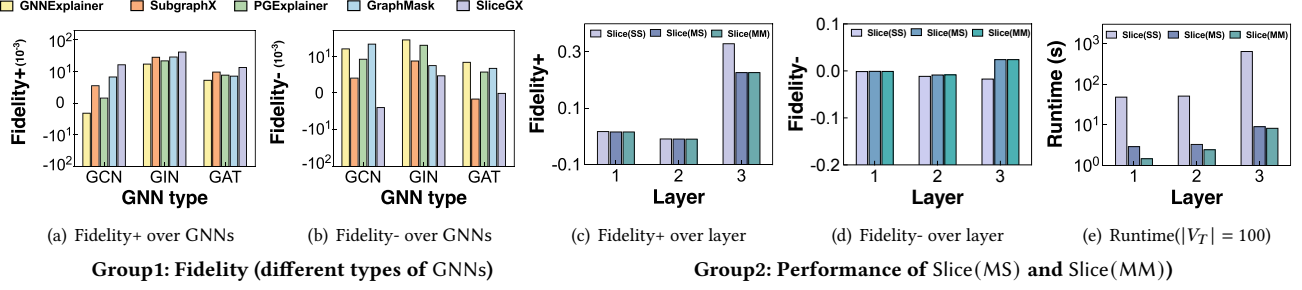


Figure 8: Additional experimental results

Table 2: Statistics and properties of datasets

Category	Dataset	# nodes	# edges	# node features	# classes
Synthetic	BA-shapes	700	4110	10	4
	Tree-Cycles	871	1950	10	2
Real-world	Cora	2708	10556	1433	7
	Coauthor CS	18333	163788	6805	15
	YelpChi	45954	7693958	32	2
	Amazon	1569960	264339468	200	107

D Datasets

We adopt six benchmark graph datasets [14], as summarized in Table 2. (1) **BA-shapes (BA)** [14] is a synthetic Barabási-Albert (BA) graphs [14]. The graph is generated by augmenting a set of nodes with structured subgraphs (motifs). A benchmark task for BA is node classification, where the nodes are associated with four numeric classes. Since the dataset does not originally provide node features, we follow PGExplainer [27] to process the dataset and derive the node features as 10-dimensional all-one vectors. (2) **Tree-Cycles (TR)** [14] are synthetic graphs obtained by augmenting 8-level balanced binary trees with six-node cycle motifs. The dataset includes two classes: one for the nodes in the base graph and another for the nodes within the defined motifs. Similar to BA-shapes, we process the dataset to derive the node features. (3) **Cora (CO)** [25] is a widely-used citation network. The nodes represent research papers, while the edges correspond to citation relationships. Each node is labeled with a topic for multi-label classification analysis. (4) **Coauthor CS [35] (CS)** is a co-authorship graph based on the Microsoft Academic Graph. The nodes include authors and papers, interlinked with co-authorship relations. The node features represent paper keywords, and each author is assigned a class label indicating his or her most active field of study. (5) **YelpChi(YC)** [12] (CS) is a binary classification dataset designed for spam reviews detection. It consists of product reviews

categorized as “Benign User” or “Fraudster”. Each node represents a review, while edges capture relationships between reviews, such as shared users, products, re-view text, or temporal connections. (6) **Amazon [45] (AM)** is a co-purchasing network, where each node is a product, and an edge between two products means they are co-purchased. The text reviews of the products are processed to generate 4-gram features, which are reduced to 200-dimensional vectors using SVD, serving as the node features. The labels represent product categories, e.g., books, movies.

E Additional Experimental Results

E.1 Different types of GNNs. We showcase that SliceGX can be also applied to different types of GNNs, including GCN [25], GIN [39], and GAT [37]. In experimental setting, the hyperparameters of GIN, including the learning rate and training epochs, closely follow the setting of GCN. For GAT, we use 10 attention heads with 10 dimensions each, and thus 100 hidden dimensions.

E.2 Performance of Slice(MS) and Slice(MM). We evaluated the overall performance and efficiency over layers by two extended algorithms of SliceGX (Slice(SS)): Slice(MS) and Slice(MM). Figs. 8(c) and 8(d) illustrate the overall performance including fidelity+ and fidelity-, and Figure 8(e) presents the time for generating explanations over layers. The results indicate that compared to SliceGX, Slice(MS) achieves faster generation speed while almost maintaining original performance due to its global updating approach. Meanwhile, Slice(MM) utilizes a “hop jumping” process and eliminates some redundant greedy selection steps, which is the most time-efficient.

This is a desirable property for online explanation generation of GNN applications over data streams such as traffic analysis, social networks, and transaction networks.